

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №2

по дисциплине: Компьютерные сети

тема: «Программирование протоколов IPX/SPX с использованием
библиотеки Winsock»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей
Александрович

Проверил:
Рубцов Константин Анатольевич

Белгород 2025 г.

Вариант 2.

Цель работы: изучить протоколы IPX/SPX, основные функции библиотеки Winsock и разработать программу для приема/передачи пакетов.

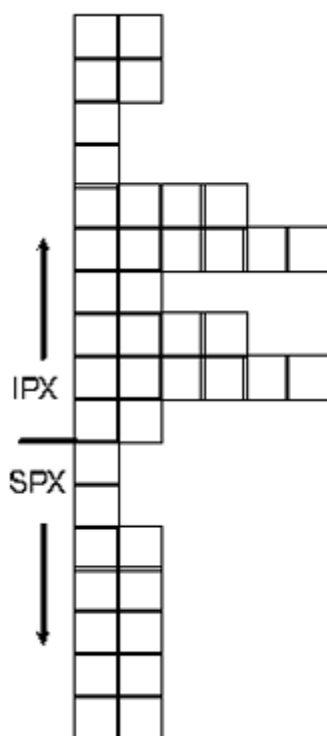
Краткие теоретические сведения

Протокол IPX (Internetwork Packet Exchange) является оригинальным протоколом сетевого уровня стека Novell, разработанным в начале 80-х годов на основе протокола Internetwork Datagram Protocol (IDM) компании Xerox [1, 3, 7].

Для надежной передачи пакетов используется протокол транспортного уровня SPX (Sequenced Packet Exchange), который работает с установлением соединения и восстанавливает пакеты при их потере или повреждении. Если по каким-то причинам пакет не дошел до получателя, выполняется его повторная передача. Следовательно, последовательность отправления совпадает с последовательностью получения пакетов. Обмен пакетами на уровне сеанса связи реализован с помощью протокола SPX, который построен на базе IPX.

Поле **ConnControl** представляет собой как набор битовых флагов, управляющих передачей данных по каналу SPX. Поле **DataStreamType** состоит из однокбитовых флагов, которые используются для классификации данных, передаваемых или принимаемых при помощи протокола SPX.

Поле **SourceConnID** содержит номер канала связи передающей программы, присвоенный драйвером SPX при создании канала связи. Этот номер должен указываться функции передачи пакета средствами SPX.



- Checksum** - контрольная сумма
- Length** - общая длина пакета
- TransportControl** - счетчик пройденных мостов
- PacketType** - тип пакета
- DestNetwork** - номер сети получателя пакета
- DestNode** - адрес станции-получателя
- DestSocket** - сокет программы-получателя
- SourceNetwork** - номер сети отправителя пакета
- SourceNode** - адрес станции-отправителя
- SourceSocket** - сокет программы-отправителя
- ConnControl** - управление потоком данных
- DataStreamType** - тип данных в пакете
- SourceConnID** - идентификатор канала отправителя
- DestConnID** - идентификатор канала получателя
- SeqNumber** - счетчик переданных пакетов
- AckNumber** - номер следующего пакета
- AllocNumber** - количество буферов для приема

Поле **DestConnID** содержит номер канала связи принимающей стороны. Так как все пакеты приходят на один номер сокета и могут принадлежать разным каналам связи (на одном сокете можно открыть несколько каналов связи), необходимо классифицировать приходящие пакеты по номеру

канала связи.

Поле **SeqNumber** содержит счетчик пакетов, переданных по каналу в одном направлении. На каждой стороне канала используется свой счетчик. После достижения значения FFFFh счетчик сбрасывается в нуль, после чего процесс счета продолжается. Содержимым этого поля управляет драйвер SPX, поэтому программа не должна менять его значение.

Поле **AckNumber** содержит номер следующего пакета, который должен быть принят драйвером SPX. Содержимым этого поля управляет драйвер SPX, поэтому программа не должна менять его значение.

Поле **AllocNumber** содержит количество буферов, распределенных программой для приема пакетов. Содержимым этого поля управляет драйвер SPX, поэтому программа не должна менять его значение [4].

Windows Sockets API (WSA) (сокр. Winsock) – техническая спецификация, которая определяет, как сетевое программное обеспечение Windows будет получать доступ к сетевым сервисам [3].

Winsock – это интерфейс сетевого программирования для Microsoft Windows.

Функция **WSAStartup** (WORD wVersionRequested, LPWSADATA lpWSADATA) инициализирует библиотеку Winsock. В случае успеха возвращает 0. Дальше можно использовать любые остальные функции этой библиотеки, иначе возвращает код возникшей ошибки. WwVersionRequested – это необходимая минимальная версия библиотеки, при присутствии которой приложение будет корректно работать. Младший байт содержит номер версии, а старший – номер ревизии. LpWSADATA – структура, в которую возвращается информация по инициализированной библиотеке (статус, версия и т.д.).

Функция **WSAGetLastError** (void) возвращает код ошибки, возникшей при выполнении последней операции. После работы с библиотекой, её необходимо выгрузить из памяти.

Функция **WSACleanup** (void) осуществляет очистку памяти, занимаемой библиотекой Winsock. Функция деинициализирует библиотеку Winsock и возвращает 0, если операция была выполнена успешно, иначе возвращает SOCKET_ERROR. Расширенный код ошибки можно получить при помощи функции WSAGetLastError.

Функция **socket** (int af, int type, int protocol) возвращает либо дескриптор созданного сокета, либо ошибку INVALID_SOCKET. Расширенный код ошибки можно получить при помощи функции WSAGetLastError.

Чтобы работать дальше с созданным сокетом его нужно привязать к какому-нибудь локальному адресу и порту. Этим занимается функция **bind** (SOCKET s, const struct sockaddr FAR* name, int namelen). Здесь s – дескриптор сокета, который данная функция именуется; name – указатель на структуру имени сокета; namelen – размер, в байтах, структуры name.

Функция **listen** (SOCKET s, int backlog) переводит сокет в состояние “прослушивания” (для протокола SPX). Здесь s – дескриптор сокета; backlog – это максимальный размер очереди входящих сообщений на соединение.

Функция **connect** (SOCKET s, const struct sockaddr FAR* name, int namelen) используется процессом-клиентом для установления связи с сервером по протоколу SPX. В случае успешного установления соединения connect возвращает 0, иначе SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Функция **accept** (SOCKET s, struct sockaddr FAR* addr, int FAR* addrlen) используется для принятия связи на сокет. Здесь s – дескриптор сокета; addr – указатель на структуру sockaddr; addrlen – размер структуры addr. Сокет должен быть уже слушающим в момент вызова функции. Если сервер устанавливает связь с клиентом, то данная функция возвращает новый сокет-дескриптор, через который и производит общение клиента с сервером. Пока устанавливается связь клиента с сервером, функция блокирует другие запросы связи с данным сервером, а после установления связи “прослушивание” запросов возобновляется [8].

В случае автоматического распределения адресов и портов узнать какой адрес и порт присвоен сокету можно при помощи функции **getsockname** (SOCKET s, struct sockaddr FAR* name, int FAR* namelen). Здесь s — дескриптор сокета; name — структура sockaddr, в 24 которую система поместит данные; namelen — размер, в байтах, структуры name. Если операция выполнена успешно, возвращает 0, иначе возвращает SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Передача данных по протоколу IPX осуществляется с помощью функции **sendto** (SOCKET s, const char FAR * buf, int len, int flags, const struct sockaddr FAR * to, int tolen). Здесь s - дескриптор сокета; buf - указатель на буфер с данными, которые необходимо переслать; len - размер (в байтах) данных, которые содержатся по указателю buf; flags - совокупность флагов, определяющих, каким образом будет произведена передача данных; to - указатель на структуру sockaddr, которая содержит адрес сокета-приёмника; tolen - размер структуры to. Если операция выполнена успешно, возвращает количество переданных байт, иначе возвращает SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Передача данных по протоколу SPX осуществляется с помощью функции **send** (SOCKET s, const char FAR * buf, int len, int flags). Здесь s - дескриптор сокета; buf - указатель на буфер с данными, которые необходимо переслать; len - размер (в байтах) данных, которые содержатся по указателю buf; flags - совокупность флагов, определяющих, каким образом будет произведена передача данных. Если операция выполнена успешно, возвращает количество переданных байт, иначе возвращает SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Прием данных по протоколу IPX осуществляется с помощью функции **recvfrom** (SOCKET s, char FAR* buf, int len, int flags, struct sockaddr FAR* from, int FAR* fromlen). Если операция выполнена успешно, возвращает количество полученных байт, иначе возвращает SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Прием данных по протоколу SPX осуществляется с помощью

функции **recv** (SOCKET s, char FAR* buf, int len, int flags). Если операция выполнена успешно, возвращает количество полученных байт, иначе возвращает SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Функция **closesocket**(SOCKET s) служит для закрытия сокета. Возвращает 0, если операция была выполнена успешно, иначе возвращает SOCKET_ERROR и номер ошибки можно получить при помощи функции WSAGetLastError.

Листинги исходного кода:

server_ipx.c:

```
#define CLIENT_REQUEST_SIZE 512
#define IMAGE_PART_SIZE 400
#define IPX_SOCKET (0x8060)

SOCKADDR_IPX name = {};

FILE *source;

void send_loop(SOCKET socket_server)
{
    bool should_run = true;
    int bytes_read;

    while (should_run)
    {
        std::string input;
        std::cout << "Input Y to send file\nType anything else to stop server:
";

        std::cout.flush();
        std::cin >> input;

        if (input != "Y")
        {
            break;
        }

        SOCKADDR_IPX client_sockaddr = {};
        client_sockaddr.sa_family = AF_IPX;
        memset(client_sockaddr.sa_netnum, 0, 4);
        memset(client_sockaddr.sa_nodenum, 0xFF, 6);
        client_sockaddr.sa_socket = htons(IPX_SOCKET);

        int client_sockaddr_size = sizeof(client_sockaddr);

        int packages_success = 0, packages_error = 0;
        fseek(source, 0, SEEK_SET);

        auto start_send_time = std::chrono::high_resolution_clock::now();
        char image_buffer[IMAGE_PART_SIZE];

        while ((bytes_read = fread(image_buffer, sizeof(char), IMAGE_PART_SIZE,
source)))
        {
            if (sendto(socket_server,
                image_buffer,
                bytes_read,
                0,
                (sockaddr *)&client_sockaddr,
                client_sockaddr_size) == SOCKET_ERROR)
            {
                packages_error++;
            }
        }
    }
}
```

```

        }
        else
        {
            packages_success++;
        }
    }
    auto end_send_time = std::chrono::high_resolution_clock::now();

    auto send_time =
std::chrono::duration_cast<std::chrono::milliseconds>(end_send_time -
start_send_time).count() / 1000.0;
    std::cout << "Image sent\nSuccessfully sent: " << packages_success <<
"\nFailed to send: " << packages_error << "\nTime: " << send_time << " s." <<
std::endl;
    }
}

int main()
{
    source = fopen("image.jpg", "rb");
    if (!source)
    {
        std::cerr << "Image not found!" << std::endl;
        return 1;
    }

    WORD wVersionRequested;
    WSADATA wsaData;

    wVersionRequested = MAKEWORD(2, 0);

    if (WSAStartup(wVersionRequested, &wsaData))
    {
        std::cerr << "WSAStartup failed with error: " << WSAGetLastError() <<
std::endl;
        return 1;
    }

    SOCKET socket_server = socket(
        AF_IPX,
        SOCK_DGRAM,
        NSPROTO_IPX);

    name.sa_family = AF_IPX;

    if (bind(socket_server, (sockaddr *)&name, sizeof(name)))
    {
        std::cerr << "Bind failed with error: " << WSAGetLastError() <<
std::endl;
        return 1;
    }

    int namelen = sizeof(name);

```

```

    getsockname(socket_server, (sockaddr *)&name, &namelen);
    std::cout << "Server is initd" << std::endl;
    printf("Net number: %02hhx %02hhx %02hhx %02hhx\n", name.sa_netnum[0],
name.sa_netnum[1], name.sa_netnum[2], name.sa_netnum[3]);
    printf("Node number: %02hhx %02hhx %02hhx %02hhx %02hhx %02hhx\n",
name.sa_nodenum[0], name.sa_nodenum[1], name.sa_nodenum[2], name.sa_nodenum[3],
name.sa_nodenum[4], name.sa_nodenum[5]);
    std::cout << "Socket num: " << htons(name.sa_socket) << std::endl;

    bool broadcast = true;
    if (setsockopt(socket_server, SOL_SOCKET, SO_BROADCAST, (char *)&broadcast,
sizeof(broadcast)) == SOCKET_ERROR)
    {
        std::cerr << "Unable to set broadcast" << std::endl;
        closesocket(socket_server);
        WSACleanup();
        return 1;
    }

    send_loop(socket_server);

    if (WSACleanup())
    {
        std::cerr << "WSACleanup failed with error: " << WSAGetLastError() <<
std::endl;
        return 1;
    }

    closesocket(socket_server);

    std::cout << "Server closed" << std::endl;

    return 0;
}

```

client_ipx.c:

```

#define IPX_SOCKET (0x8060)
#define CLIENT_REQUEST_SIZE 512
#define IMAGE_PART_SIZE 400

SOCKADDR_IPX name = {};
SOCKADDR_IPX server_sockaddr = {};

void record_loop(SOCKET socket_descriptor, FILE *dist)
{
    std::cout << "Starting file accept..." << std::endl;
    char *buffer = (char *)malloc(sizeof(char) * IMAGE_PART_SIZE);
    int bytes_received;
    while ((bytes_received = recvfrom(socket_descriptor, buffer, sizeof(char) *
IMAGE_PART_SIZE, 0, nullptr, nullptr)) != SOCKET_ERROR)
    {
        fwrite(buffer, sizeof(char), bytes_received, dist);
    }
}

```

```

    free(buffer);
}

int main()
{
    char filename[20];

    WORD wVersionRequested;
    WSADATA wsaData;

    wVersionRequested = MAKEWORD(2, 0);

    if (WSAStartup(wVersionRequested, &wsaData))
    {
        printf("WSAStartup failed with error: %d\n", WSAGetLastError());
        return 1;
    }

    SOCKET socket_descriptor = socket(
        AF_IPX,
        SOCK_DGRAM,
        NSPROTO_IPX);
    name.sa_family = AF_IPX;
    name.sa_socket = htons(IPX_SOCKET);

    if (bind(socket_descriptor, (sockaddr *)&name, sizeof(name)))
    {
        printf("bind failed with error: %d\n", WSAGetLastError());
        return 1;
    }

    int namelen = sizeof(name);
    getsockname(socket_descriptor, (sockaddr *)&name, &namelen);

    std::cout << "Client is initied:\n";
    printf("Net number: %02hhx %02hhx %02hhx %02hhx\n", name.sa_netnum[0],
name.sa_netnum[1], name.sa_netnum[2], name.sa_netnum[3]);
    printf("Node number: %02hhx %02hhx %02hhx %02hhx %02hhx %02hhx\n",
name.sa_nodenum[0], name.sa_nodenum[1], name.sa_nodenum[2], name.sa_nodenum[3],
name.sa_nodenum[4], name.sa_nodenum[5]);
    std::cout << "Socket num: " << htons(name.sa_socket) << std::endl;

    int timeout_time = 15000;

    if (setsockopt(socket_descriptor, SOL_SOCKET, SO_RCVTIMEO, (char
*)&timeout_time, sizeof(timeout_time)) == SOCKET_ERROR)
    {
        printf("Unable to set timeout: %d\n", WSAGetLastError());
        return 1;
    }

    srand(time(NULL));

```

```

    sprintf(filename, "img%d.jpg", rand() % 100 + 1);
    FILE *dist = fopen(filename, "wb");

    record_loop(socket_descriptor, dist);

    fflush(dist);
    fclose(dist);

    if (WSACleanup())
    {
        printf("WSACleanup failed with error: %d\n", WSAGetLastError());
        return 1;
    }

    closesocket(socket_descriptor);

    std::cout << "A file was saved at " << filename << "\nPress any button to
exit" << std::endl;
    getchar();

    std::cout << "Client close" << std::endl;

    return 0;
}

```

server_spx.c:

```

#define CLIENT_REQUEST_SIZE 512
#define IMAGE_PART_SIZE 400
#define BACKLOG_MAX_SIZE 10
#define IPX_SOCKET (0x8060)
#define SPX_SOCKET (0x5E86)
#define CHECK_MSG_QUESTION "What are u say about my momo`s sports steering
wheel?"
#define CHECK_MSG_ANSWER "I`dont what u... uh, just go"

bool should_run = true;

void* ping_message_task(void* arg)
{
    SOCKADDR_IPX name = {};
    SOCKET socket_descriptor = socket(
        AF_IPX,
        SOCK_DGRAM,
        NSPROTO_IPX);
    name.sa_family = AF_IPX;
    name.sa_socket = htons(IPX_SOCKET);

    if (bind(socket_descriptor, (sockaddr *)&name, sizeof(name)))
    {
        printf("bind failed with error: %d\n", WSAGetLastError());
        return nullptr;
    }
}

```

```

int namelen = sizeof(name);
getsockname(socket_descriptor, (sockaddr *)&name, &namelen);
std::cout << "Ping server is initied\n";
int timeout_time = 1000;
setsockopt(
    socket_descriptor,
    SOL_SOCKET,
    SO_RCVTIMEO,
    (char *)&timeout_time,
    sizeof(timeout_time));

char *accept_client_data = (char *)malloc(sizeof(char) *
CLIENT_REQUEST_SIZE);

while (should_run)
{
    SOCKADDR_IPX client_sockaddr = {};
    client_sockaddr.sa_family = AF_IPX;
    int client_sockaddr_size = sizeof(client_sockaddr);

    if (recvfrom(
        socket_descriptor,
        accept_client_data,
        sizeof(char) * CLIENT_REQUEST_SIZE,
        0,
        (sockaddr *)&client_sockaddr,
        &client_sockaddr_size) == SOCKET_ERROR)
    {
    }
    else if (strcmp(accept_client_data, CHECK_MSG_QUESTION) == 0)
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        char answer[] = CHECK_MSG_ANSWER;
        sendto(socket_descriptor,
            answer,
            sizeof(answer),
            0,
            (sockaddr *)&client_sockaddr,
            client_sockaddr_size);
    }
}

closesocket(socket_descriptor);
free(accept_client_data);
return nullptr;
}

void* serve_client_task(void* arg)
{
    SOCKET client_socket = (SOCKET) arg;

    printf("Serving a new request\n");
    char client_buffer[CLIENT_REQUEST_SIZE] = {};

```

```

    if (recv(client_socket, client_buffer, sizeof(client_buffer), 0) ==
SOCKET_ERROR)
    {
        printf("Failed to get message for client: %d\n", WSAGetLastError());
        closesocket(client_socket);
        return nullptr;
    }

    printf("A request for file '%s'\n", client_buffer);

    FILE *source = fopen(client_buffer, "rb");
    if (!source)
    {
        std::cerr << "File not found" << std::endl;
        closesocket(client_socket);
        return nullptr;
    }

    std::cout << "Everything is OK, sending a file..." << std::endl;

    auto start_send_time = std::chrono::high_resolution_clock::now();
    int bytes_read = 0;
    char image_buffer[IMAGE_PART_SIZE];
    while ((bytes_read = fread(image_buffer, sizeof(char), IMAGE_PART_SIZE,
source)))
    {
        if (send(client_socket,
                image_buffer,
                bytes_read,
                0) == SOCKET_ERROR)
        {
            std::cerr << "Failed to send package: %d" << WSAGetLastError() <<
std::endl;
            closesocket(client_socket);
            fclose(source);
            return nullptr;
        }
    }

    auto end_send_time = std::chrono::high_resolution_clock::now();

    auto send_time =
std::chrono::duration_cast<std::chrono::milliseconds>(end_send_time -
start_send_time).count() / 1000.0;
    std::cout << "File sent successfully\nTime:" << send_time << " s." <<
std::endl;

    closesocket(client_socket);
    fclose(source);
    return nullptr;
}

void* mainloop_task(void* arg)

```

```

{
    SOCKADDR_IPX name = {};
    SOCKET socket_descriptor = socket(
                                AF_IPX,
                                SOCK_SEQPACKET,
                                NSPROTO_SPX
                                );

    name.sa_family = AF_IPX;
    name.sa_socket = htons(SPX_SOCKET);

    if (bind(socket_descriptor, (sockaddr*)&name, sizeof(name)) == SOCKET_ERROR)
    {
        printf("bind failed with error: %d\n", WSAGetLastError());
    }

    int namelen = sizeof(name);
    getsockname(socket_descriptor, (sockaddr *)&name, &namelen);
    std::cout << "Server is initied\n";
    printf("Net number: %02hhx %02hhx %02hhx %02hhx\n", name.sa_netnum[0],
name.sa_netnum[1], name.sa_netnum[2], name.sa_netnum[3]);
    printf("Node number: %02hhx %02hhx %02hhx %02hhx %02hhx %02hhx\n",
name.sa_nodenum[0], name.sa_nodenum[1], name.sa_nodenum[2], name.sa_nodenum[3],
name.sa_nodenum[4], name.sa_nodenum[5]);
    std::cout << "Socket num: " << htons(name.sa_socket) << "\n";
    std::cout.flush();

    if (listen(socket_descriptor, BACKLOG_MAX_SIZE) == SOCKET_ERROR)
    {
        printf("Couldn't startup server (listen failed): %d\n",
WSAGetLastError());
        return nullptr;
    }

    while (should_run)
    {
        std::cout << "Awaiting for client connections..." << std::endl;
        SOCKADDR_IPX clientAddr;
        int clientAddrSize = sizeof(clientAddr);

        SOCKET client_descriptor = accept(
            socket_descriptor,
            (sockaddr *)&clientAddr,
            &clientAddrSize);

        if (client_descriptor != INVALID_SOCKET)
        {
            pthread_t serve_client_thread;
            pthread_create(&serve_client_thread, nullptr, serve_client_task,
(void*) client_descriptor);
            pthread_detach(serve_client_thread);
        }
    }
}

```

```

        closesocket(socket_descriptor);
        return nullptr;
    }

int main()
{
    WORD wVersionRequested;
    WSADATA wsaData;

    wVersionRequested = MAKEWORD(2, 2);

    if (WSAStartup(wVersionRequested, &wsaData))
    {
        printf("WSAStartup failed with error: %d\n", WSAGetLastError());
        return 1;
    }

    pthread_t ping_thread, mainloop_thread;
    pthread_create(&ping_thread, nullptr, ping_message_task, nullptr);
    pthread_create(&mainloop_thread, nullptr, mainloop_task, nullptr);

    getchar();

    should_run = false;
    std::cout << "Server close! Quitting threads..." << std::endl;
    pthread_join(ping_thread, nullptr);
    pthread_join(mainloop_thread, nullptr);

    WSACleanup();

    return 0;
}

```

client_spx.c:

```

#define CLIENT_REQUEST_SIZE 512
#define IMAGE_PART_SIZE 400
#define BACKLOG_MAX_SIZE

#define IPX_SOCKET (0x8060)
#define SPX_SOCKET (0x5E86)

#define CHECK_MSG_QUESTION "What are u say about my momo`s sports steering wheel?"
#define CHECK_MSG_ANSWER "I`dont what u... uh, just go"

SOCKET socket_descriptor;
SOCKADDR_IPX name = {};
SOCKADDR_IPX server_sockaddr = {};

SOCKADDR_IPX server_address;

void send_request()
{

```

```

char data[CLIENT_REQUEST_SIZE] = CHECK_MSG_QUESTION;

while (true)
{
    printf("Input netnum: ");
    scanf("%02hhx %02hhx %02hhx %02hhx", server_sockaddr.sa_netnum,
server_sockaddr.sa_netnum + 1, server_sockaddr.sa_netnum + 2,
server_sockaddr.sa_netnum + 3);

    memset(server_sockaddr.sa_nodenum, 0xFF, 6);
    server_sockaddr.sa_socket = htons(IPX_SOCKET);
    server_sockaddr.sa_family = AF_IPX;

    if (sendto(socket_descriptor,
                data,
                CLIENT_REQUEST_SIZE * sizeof(char),
                0,
                (sockaddr *)&server_sockaddr,
                sizeof(server_sockaddr)) == SOCKET_ERROR)
    {
        printf("Unable to connect to server: %d\nTry again\n",
WSAGetLastError());
    }
    else
    {
        return;
    }
}

void rec_request()
{
    printf("Starting answer accept...\n");
    char *buffer = (char *)malloc(sizeof(char) * IMAGE_PART_SIZE);
    while (1)
    {
        SOCKADDR_IPX receive_name = {};
        receive_name.sa_family = AF_IPX;
        int receive_name_size = sizeof(receive_name);

        int bytes_received;
        if (bytes_received = recvfrom(
                                socket_descriptor,
                                buffer,
                                sizeof(char) * IMAGE_PART_SIZE,
                                0,
                                (SOCKADDR *)&receive_name,
                                &receive_name_size) != SOCKET_ERROR)
        {
            if (strcmp(buffer, CHECK_MSG_ANSWER) == 0)
            {
                server_address = receive_name;
                server_address.sa_socket = htons(SPX_SOCKET);
            }
        }
    }
}

```

```

        server_address.sa_family = AF_IPX;
        break;
    }
}
else
{
    break;
}
}
}

void request_file()
{
    SOCKADDR_IPX rf_name = {};

    SOCKET socket_descriptor = socket(
        AF_IPX,
        SOCK_SEQPACKET,
        NSPROTO_SPX);
    rf_name.sa_family = AF_IPX;
    if (bind(socket_descriptor, (sockaddr *)&rf_name, sizeof(rf_name)) ==
SOCKET_ERROR)
    {
        printf("bind failed with error: %d\n", WSAGetLastError());
        return;
    }

    if (connect(socket_descriptor, (sockaddr *)&server_address,
sizeof(server_address)) == SOCKET_ERROR)
    {
        printf("Failed to connect to server %d\n", WSAGetLastError());
        closesocket(socket_descriptor);
        return;
    }

    std::cout << "Server connection established\n";

    std::string filename;
    printf("Input filename: ");
    std::cin >> filename;

    FILE *save_file = fopen(filename.c_str(), "wb");

    if (!save_file)
    {
        std::cout << "Unable to open file for saving" << std::endl;
        closesocket(socket_descriptor);
        return;
    }

    if (send(socket_descriptor, filename.c_str(), filename.size() + 1, 0) ==
SOCKET_ERROR)
    {

```

```

        printf("Failed to send request for file %d\n", WSAGetLastError());
        closesocket(socket_descriptor);
        return;
    }

    int diff = 0;
    int total_diff = 0;
    int bytes_read;
    char image_buffer[IMAGE_PART_SIZE];
    while ((bytes_read = recv(socket_descriptor, image_buffer, sizeof(char) *
IMAGE_PART_SIZE, 0)) > 0)
    {
        diff += bytes_read;
        total_diff += bytes_read;

        while (diff > 1000000)
        {
            std::cout << "Accepted " << total_diff / 1024 << " KBs..." <<
std::endl;
            diff -= 1000000;
        }

        fwrite(image_buffer, sizeof(char), bytes_read, save_file);
    }

    std::cout << "File accepted succesfully! Saving result into '" << filename
<< "'" << std::endl;

    fflush(save_file);
    fclose(save_file);

    closesocket(socket_descriptor);
}

int main()
{
    char filename[20];

    WORD wVersionRequested;
    WSADATA wsaData;

    wVersionRequested = MAKEWORD(2, 0);

    if (WSAStartup(wVersionRequested, &wsaData))
    {
        printf("WSAStartup failed with error: %d\n", WSAGetLastError());
        return 1;
    }

    socket_descriptor = socket(
        AF_IPX,
        SOCK_DGRAM,
        NSPROTO_IPX);

```

```

name.sa_family = AF_IPX;

if (bind(socket_descriptor, (sockaddr *)&name, sizeof(name)))
{
    printf("bind failed with error: %d\n", WSAGetLastError());
    return 1;
}

int timeout_time = 4000;
bool allow_broadcast = true;

setsockopt(socket_descriptor, SOL_SOCKET, SO_RCVTIMEO, (char
*)&timeout_time, sizeof(timeout_time));
setsockopt(socket_descriptor, SOL_SOCKET, SO_BROADCAST, (char
*)&allow_broadcast, sizeof(allow_broadcast));

send_request();
rec_request();
request_file();

if (WSACleanup())
{
    printf("WSACleanup failed with error: %d\n", WSAGetLastError());
    return 1;
}

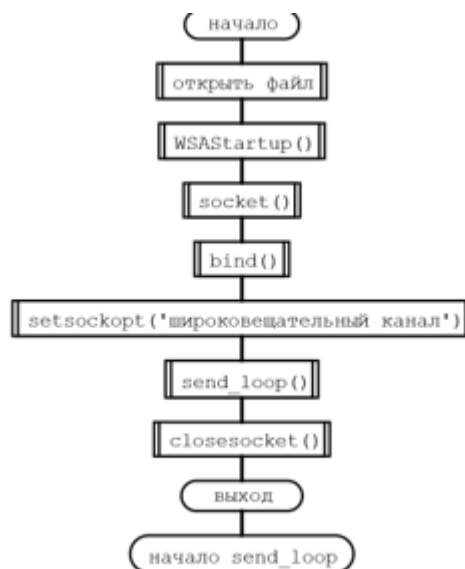
closesocket(socket_descriptor);

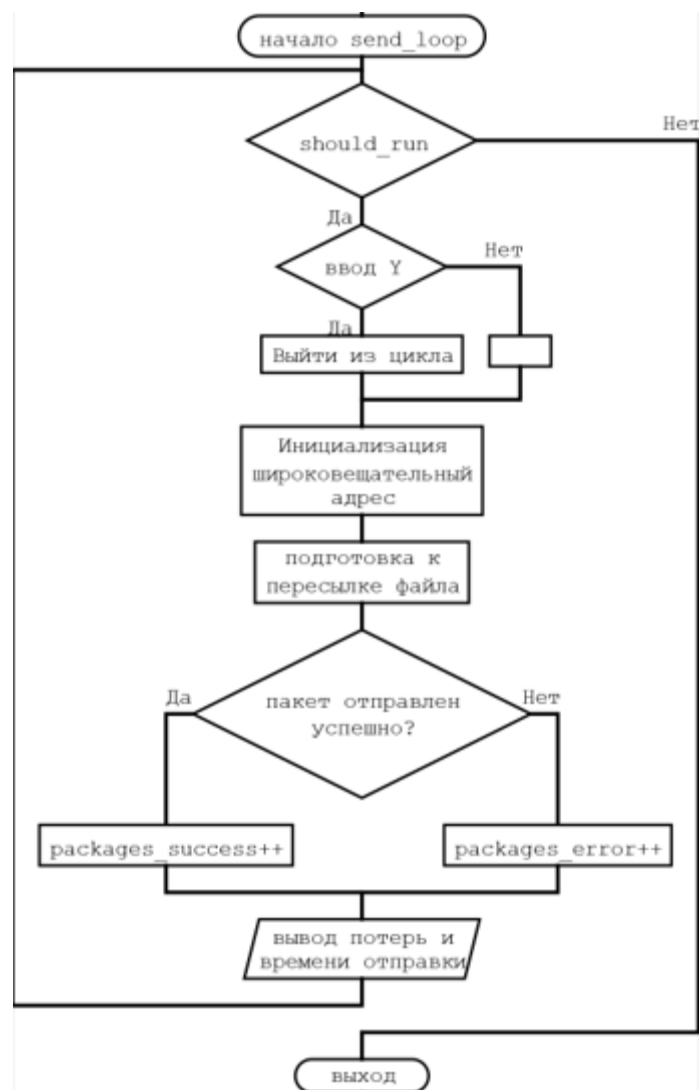
std::cout << "Have a good day!" << std::endl;

return 0;
}

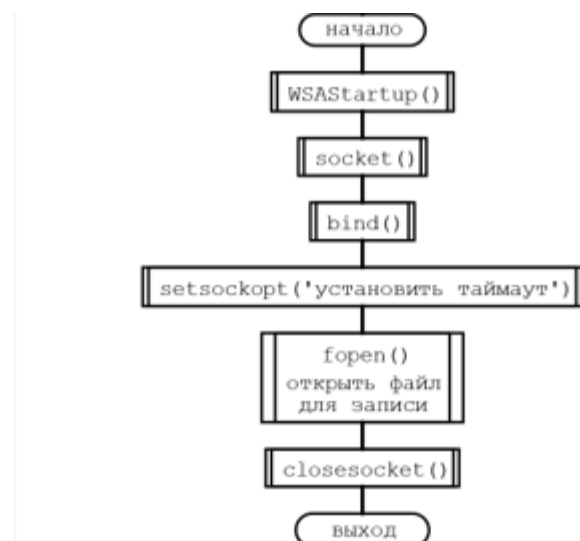
```

Блок схемы программ:
server_ipx:

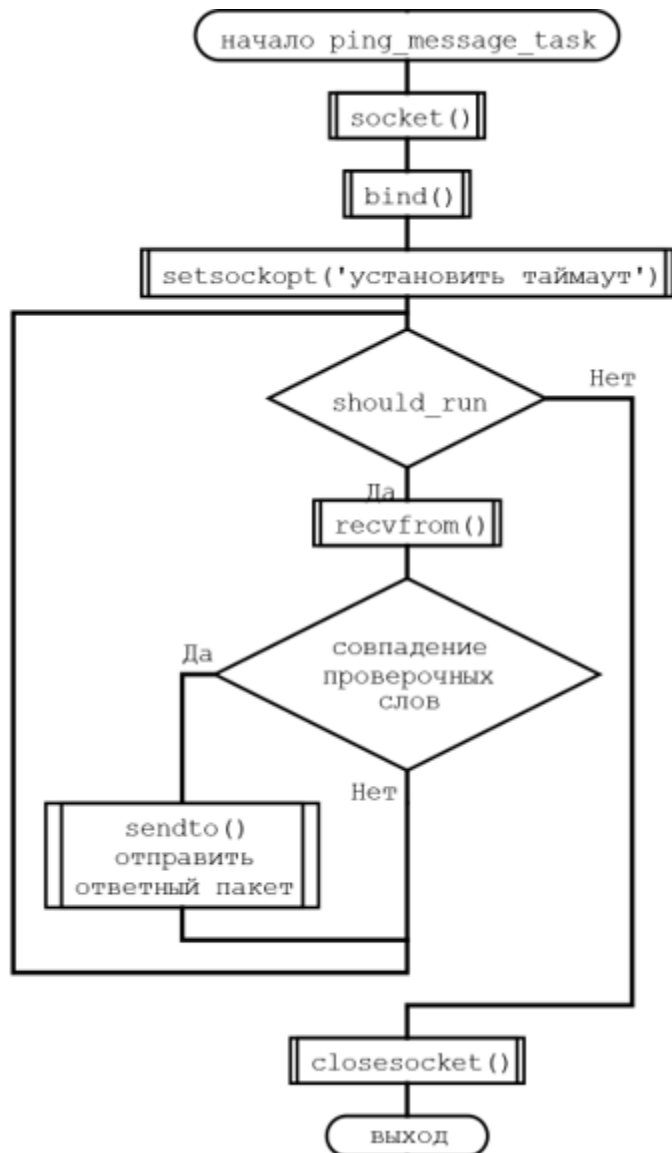


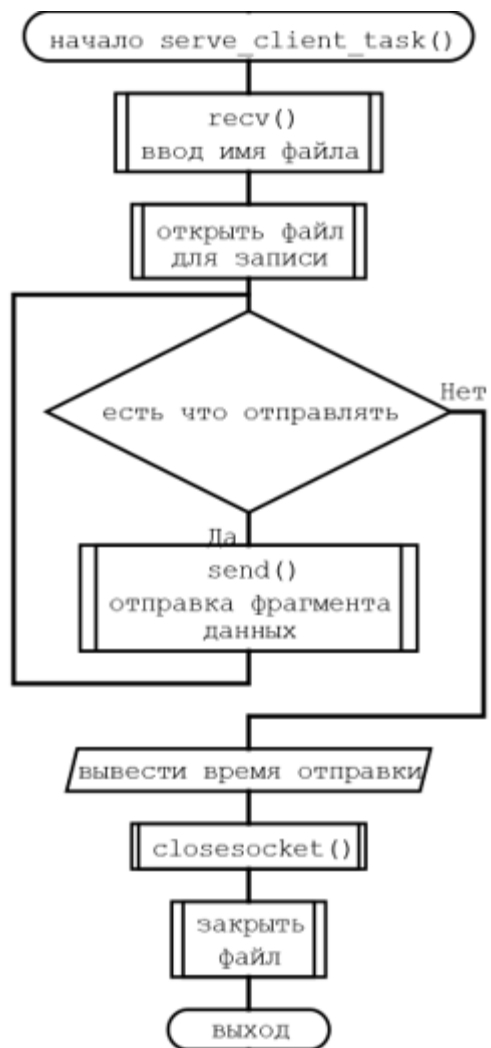
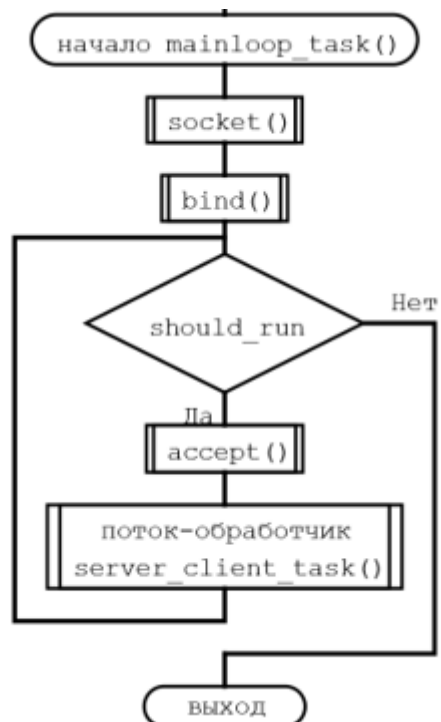


client_ipx:

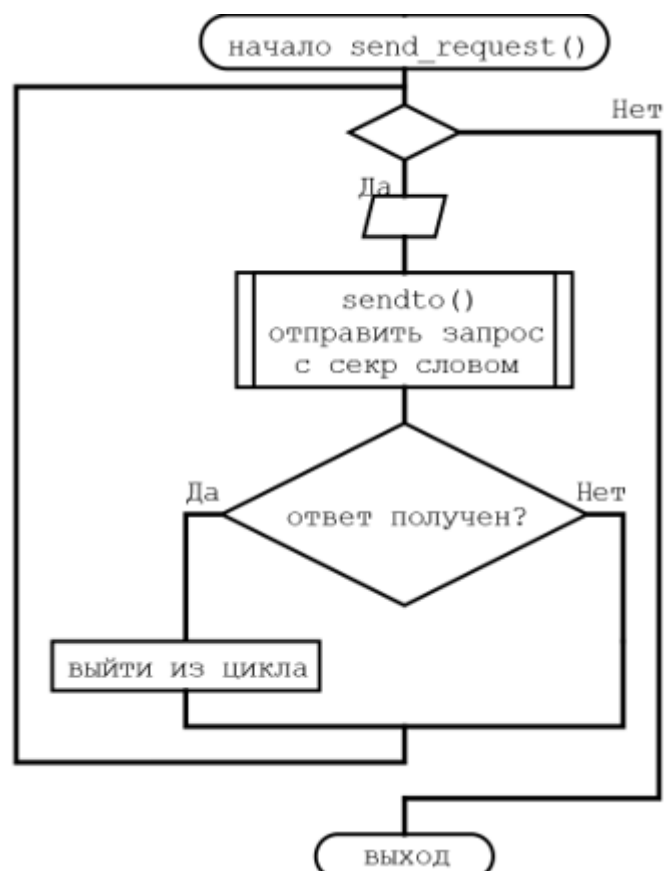
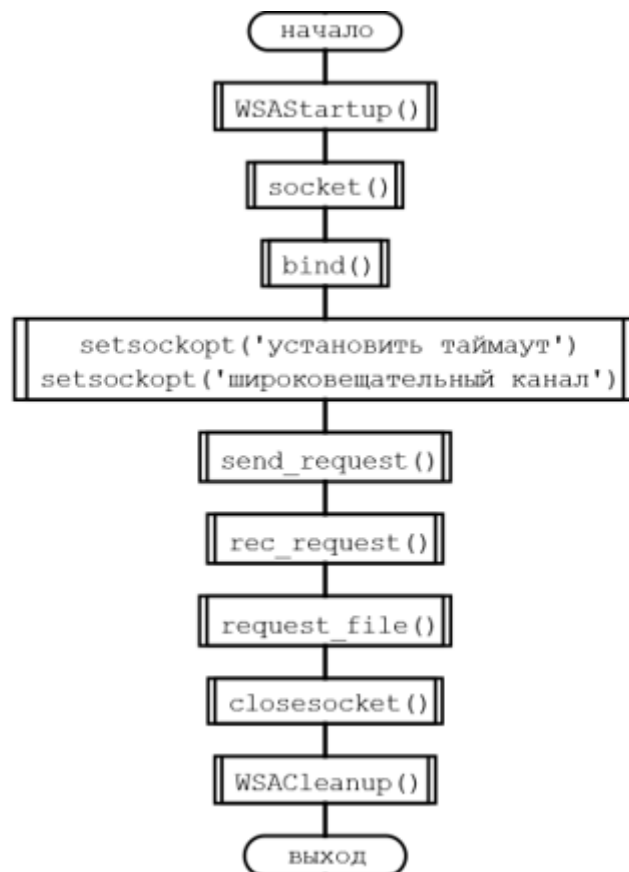


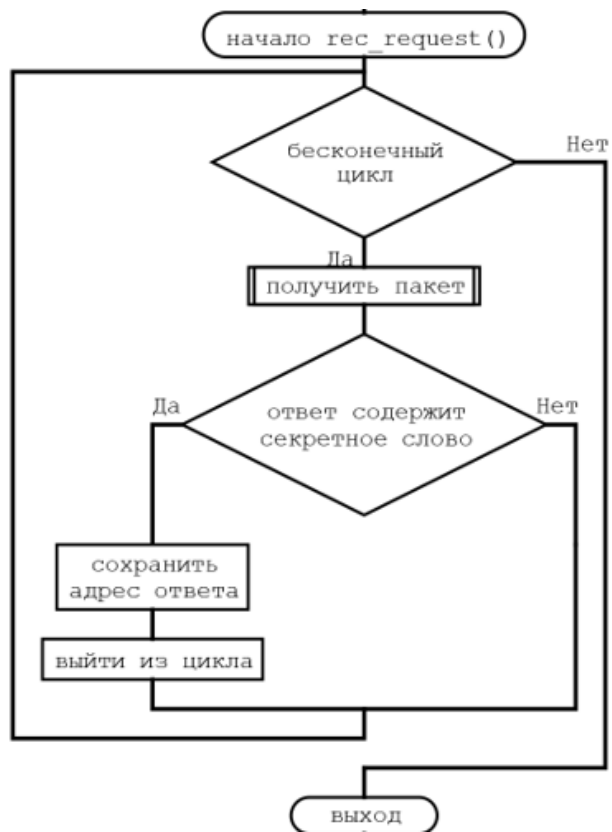
server_spx:

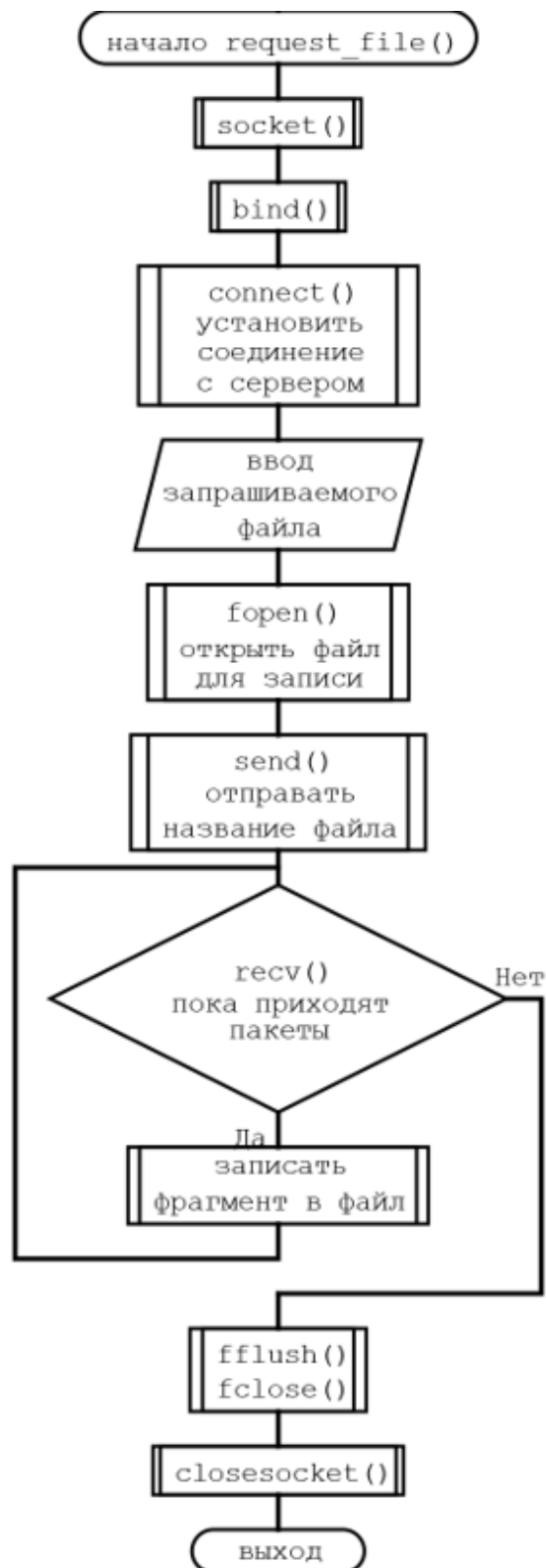




client_spx:







Анализ функционирования разработанных программ

Картинка:



Тест 1.

Протокол: ipx

Режим: на одной машине

Размер файла: 20787 Киб

Размер данных пакета: 400 байт

Предполагаемое кол-во пакетов: 53 371

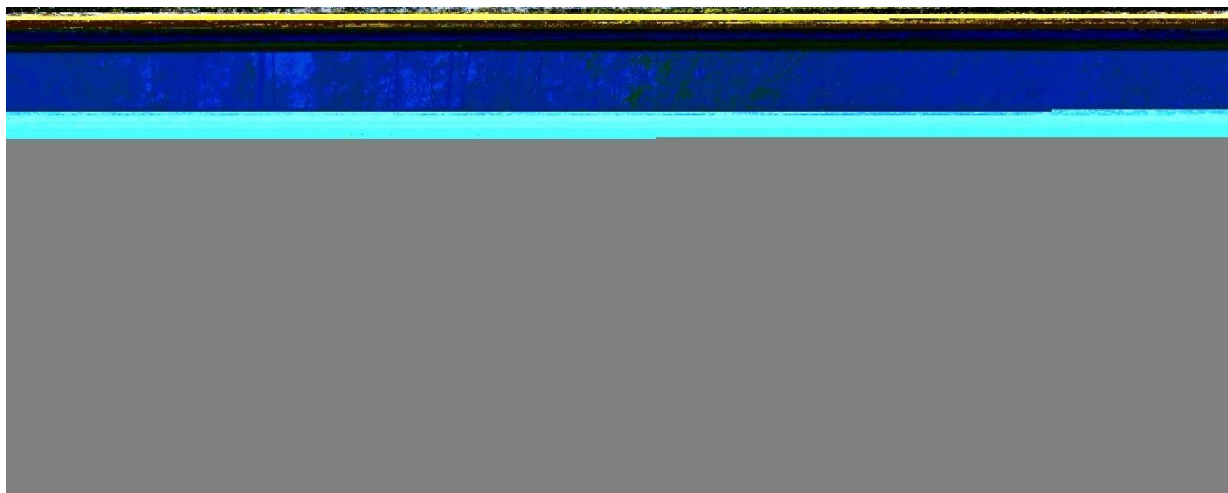
Общее время: 62,5 с

Кол-во получателей: 1

Вердикт: Неуспешная пересылка

The image shows two overlapping command prompt windows. The top window is titled 'Z:\lab2_server_ipx\bin\Debug\server_ipx.exe'. It displays the following text: 'Server is initied', 'Net number: 00 00 00 00', 'Node number: 08 00 27 fffc 4a 3c', 'Socket num: 16520', 'Input Y to send file', 'Type anything else to stop server: Y', 'Image sent', 'Successfully sent: 53371', 'Failed to send: 0', 'Time: 62.529 s.', 'Input Y to send file', and 'Type anything else to stop server:'. The bottom window is titled 'Z:\lab2_client_ipx\bin\Debug\client_ipx.exe'. It displays the following text: 'Client is initied:', 'Net number: 00 00 00 00', 'Node number: 08 00 27 fffc 4a 3c', 'Socket num: 32864', and 'Starting file accept...'. Both windows have a black background with white text.

Результат пересылки:



Тест 2.

Протокол: ipx

Режим: на нескольких машинах

Размер файла: 20787 Киб

Размер данных пакета: 400 байт

Предполагаемое кол-во пакетов: 53 371

Общее время: 74,6 с

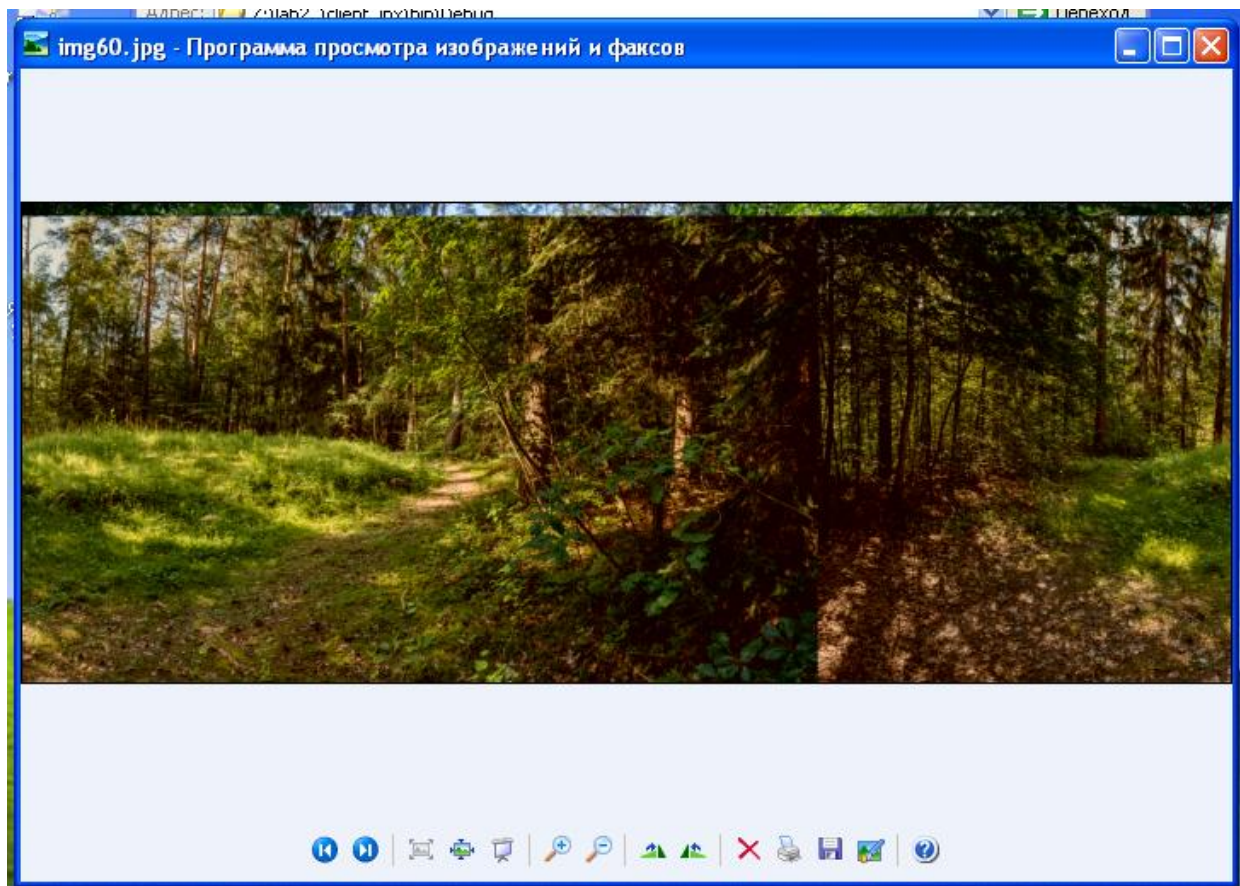
Кол-во получателей: 1

Вердикт: Неуспешная пересылка

```
Server is initied
Net number: 00 00 00 00
Node number: 08 00 27 ff9f fff6 32
Socket num: 16392
Input Y to send file
Type anything else to stop server: Y
Image sent
Successfully sent: 53371
Failed to send: 0
Time: 74.597 s.
Input Y to send file
Type anything else to stop server:

Client is initied:
Net number: 00 00 00 00
Node number: 08 00 27 fffc 4a 3c
Socket num: 32864
Starting file accept...
A file was saved at img60.jpg
Press any button to exit
```

Результат пересылки:



Тест 3.

Протокол: spx

Режим: на нескольких машинах

Размер файла: 20787 Киб

Размер данных пакета: 400 байт

Предполагаемое кол-во пакетов: 53 371

Общее время: ~162.5с

Кол-во получателей: 1

Вердикт: Успешная пересылка

A screenshot of a Windows command prompt window. The title bar reads 'Z:\lab2_server_spx\bin\Debug\server_spx.exe'. The window contains the following text:

```
Server is inited
Net number: 12 34 cd ef
Ping server is inited
Node number: 00 00 00 00 00 02
Socket num: 24198
Awaiting for client connections...
Awaiting for client connections...
Serving a new request
A request for file 'image.jpg'
Everything is OK, sending a file...
File sent successfully
Time:162.205 s.
Awaiting for client connections...
Serving a new request
A request for file 'image.png'
File not found
Awaiting for client connections...
Serving a new request
A request for file 'image.jpg'
Everything is OK, sending a file...
File sent successfully
Time:163.294 s.
```

Тест 4.

Протокол: srx

Режим: на нескольких машинах

Размер файла: 20787 Киб

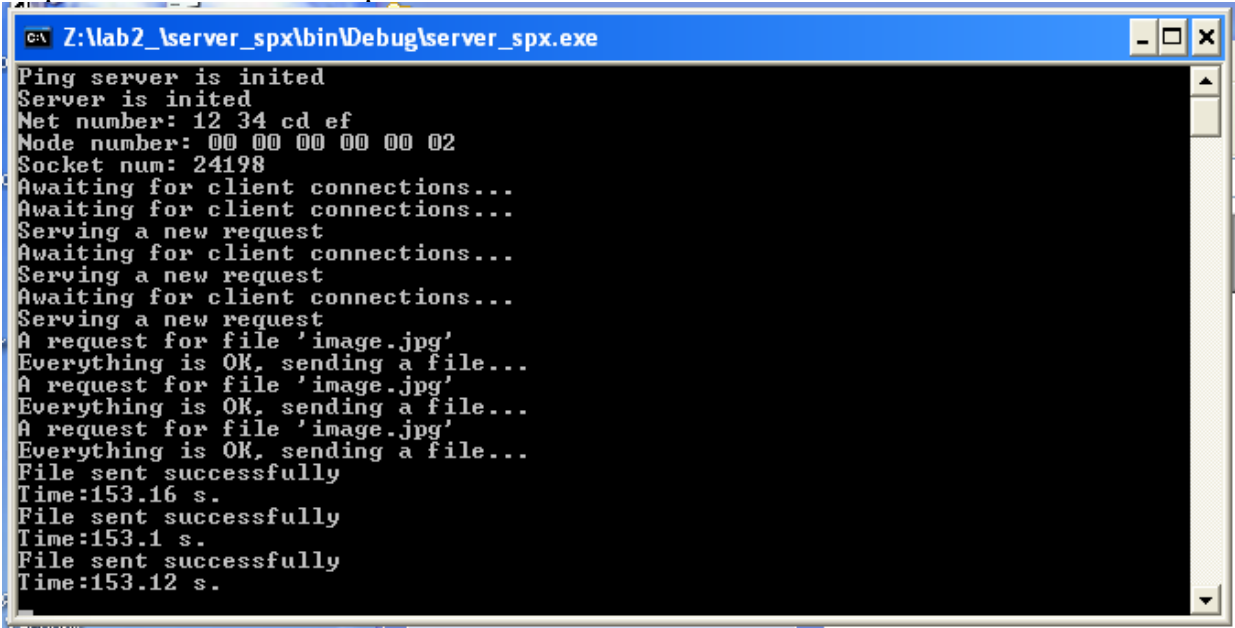
Размер данных пакета: 400 байт

Предполагаемое кол-во пакетов: 53 371

Общее время: ~153.1 с

Кол-во получателей: 3

Вердикт: Успешная пересылка

A screenshot of a Windows command prompt window. The title bar is blue and contains the text "Z:\lab2_server_spx\bin\Debug\server_spx.exe". The window has standard Windows window controls (minimize, maximize, close) on the right. The command prompt area is black with white text. The text shows the server's startup sequence, including initialization, listening for connections, and serving three requests for a file named 'image.jpg'. Each request is followed by a confirmation message and a timestamp.

```
C:\Z:\lab2_server_spx\bin\Debug\server_spx.exe
Ping server is initied
Server is initied
Net number: 12 34 cd ef
Node number: 00 00 00 00 00 02
Socket num: 24198
Awaiting for client connections...
Awaiting for client connections...
Serving a new request
Awaiting for client connections...
Serving a new request
Awaiting for client connections...
Serving a new request
A request for file 'image.jpg'
Everything is OK, sending a file...
A request for file 'image.jpg'
Everything is OK, sending a file...
A request for file 'image.jpg'
Everything is OK, sending a file...
File sent successfully
Time:153.16 s.
File sent successfully
Time:153.1 s.
File sent successfully
Time:153.12 s.
```

Вывод: в ходе работы изучены протоколы IPX/SPX, основные функции библиотеки Winsock и разработана программа для приема/передачи пакетов.